

SEG3502 – Lab 3

Angular – Composants

L'objectif de ce laboratoire est d'en savoir plus sur Angular en implémentant une application Carnet d'adresses (Address Book). Plus précisément, vous apprendrez:

- la création et la mise en œuvre de composants Angular,
- l'implémentation de formulaires basés templates,
- la communication entre composants avec des Inputs, des événements Outputs et des Observables

Le code source du laboratoire est disponible dans le référentiel Github

<https://github.com/stephanesome/angComponents.git>.

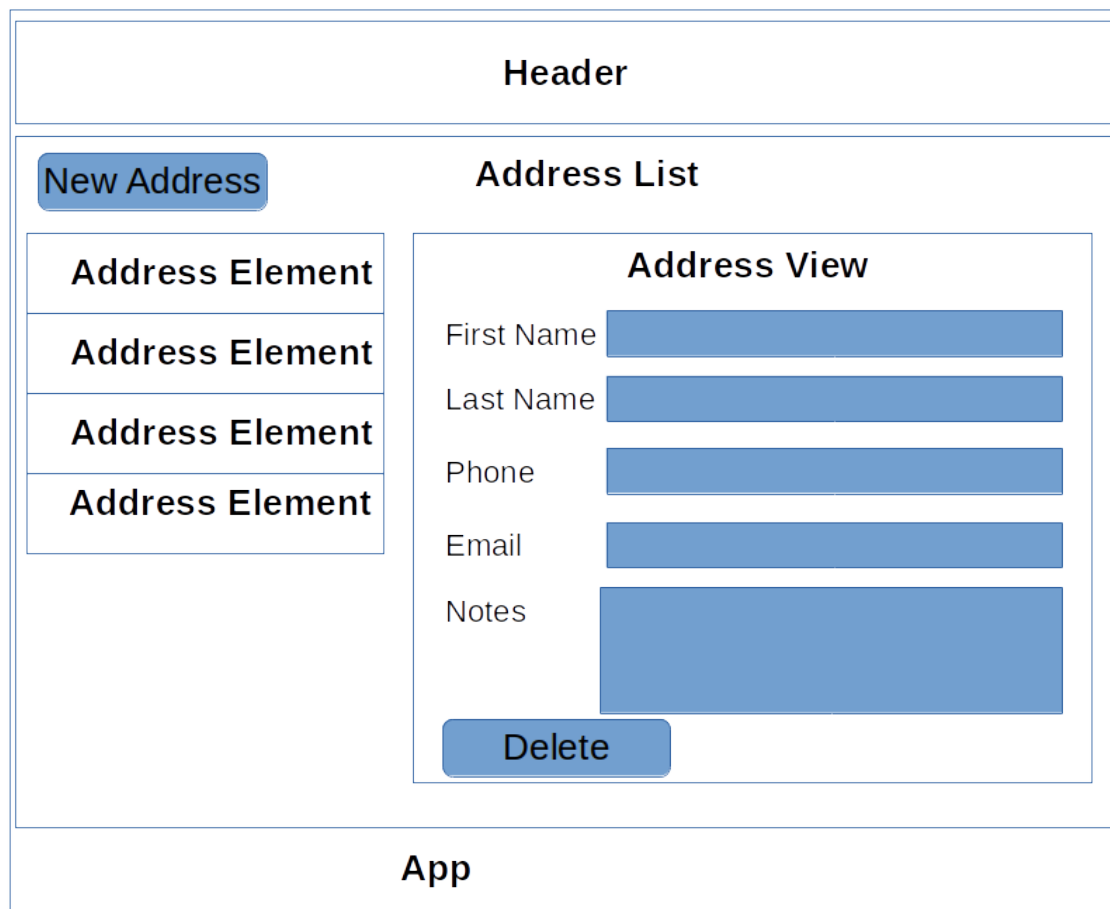
Application Address Book

Nous mettrons en œuvre une application pour enregistrer le carnet d'adresses d'un utilisateur. Les principales caractéristiques de l'application sont les suivantes.

- Afficher une liste des entrées du carnet d'adresses enregistrées.
- Ajouter des entrées au carnet d'adresses.
- Afficher les détails des entrées du carnet d'adresses. Chaque entrée comprend un prénom, un nom, un numéro de téléphone, un e-mail et des notes (first name, last name, phone number, email, notes).
- Supprimer des entrées du carnet d'adresses.

Conception des composants

La conception de l'interface est indiquée pour toute application. La conception de l'interface permet de déterminer la disposition de l'interface et de s'assurer qu'elle supporte les utilisateurs de manière efficace. Cela nous permet également de comprendre les composants nécessaires pour une application Angular. Vous trouverez ci-dessous le wireframe de l'application.



Nous identifions les composants suivants et leurs relations.

- Composant *App*, le composant racine de l'application.
- Composant *Header* sous-composant de *App*.
- Composant *Address List* sous-composant de *App*. Ce composant contient une liste d'adresses et fournit des responsabilités associées telles que l'ajout d'une adresse à la liste ou la suppression d'une adresse de la liste.
- Composant *Address Element*, sous-composant de liste d'adresses. Ce composant est un élément sélectionnable pour une entrée de carnet d'adresses. Cliquer dessus affiche les détails de l'entrée correspondante.
- Composant *Address View* sous-composant de la liste d'adresses. Ce composant permet d'afficher et de modifier une entrée du carnet d'adresses.

Configuration de l'application

Créez une nouvelle application angular (`ng new address-book --standalone`). Nous utiliserons CSS comme format de feuille de style.

Nous utiliserons la [bibliothèque CSS Angular Bootstrap](#). Exécutez la commande `ng add @ng-bootstrap/ng-bootstrap` dans le dossier racine du projet pour ajouter Bootstrap au projet..

Composant App

Le composant racine *App* contient le composant *Header* ainsi que le composant *Address List*. Spécifiez le template HTML (`src/app/app.html`) comme suit.

```
1. <app-header></app-header>
2. <div class="container">
3.   <app-address-list></app-address-list>
4. </div>
```

Composant Header

Le composant *Header* inclut uniquement la personnalisation de notre application. Exécutez la commande `ng generate component header` dans le dossier racine du projet. Angular CLI générera des fichiers pour un nouveau composant avec la sortie suivante.

```
CREATE src/app/header/header.component.css (0 bytes)
CREATE src/app/header/header.component.html (21 bytes)
CREATE src/app/header/header.component.spec.ts (628 bytes)
CREATE src/app/header/header.component.ts (275 bytes)
UPDATE src/app/app.module.ts (467 bytes)
```

Les fichiers générés incluent:

- la classe de composant dans `src/app/header/header.ts`,
- test du composant dans `src/app/header/header.spec.ts`,
- le template HTML de composant dans `src/app/header/header.css` and
- le fichier de feuille CSS du composant dans `src/app/header/header.css`.

Modifiez le template HTML comme suit

```
1. <nav class="navbar navbar-fixed-top">
2.   <div class="container-fluid">
3.     <div class="navbar-header">
4.       <a href="#" class="navbar-brand">Your Address Book</a>
5.     </div>
6.   </div>
7. </nav>
```

Composant Address List

Générez le composant de liste d'adresses (`ng generate component address-list`).

Modèle d'entrée d'adresse

Nous avons besoin d'une classe de modèle pour les entrées du carnet d'adresses. Ceci est juste une classe Typescript normale. Nous utilisons Angular CLI pour générer avec la commande `ng generate class address-list/address-entry`.

Modifiez le fichier `src/app/address-list/address-entry.ts` comme suit.

```
1. export class AddressEntry {
2.   public firstName: string;
3.   public lastName: string;
4.   public phone?: string;
5.   public email?: string;
6.   public notes?: string;
7.
8.   constructor(firstName: string, lastName: string, phone?: string, email?: string, notes?: string) {
9.     this.firstName = firstName;
10.    this.lastName = lastName;
11.    this.phone = phone;
12.    this.email = email;
13.    this.notes = notes;
14.  }
15. }
```

La définition de classe spécifie des champs d'entrée d'adresse et un constructeur.

Template du composant Address List

Le template HTML du composant *Address List* doit présenter un bouton pour ajouter une nouvelle entrée, une liste d'éléments de liste d'adresses (*Address List Element*) et un composant pour l'affichage d'adresses (*Address View*). Créez un template initial comme suit dans `src/app/address-list/address-list.html`.

```
1. <div class="row">
2.   <div class="col-md-12">
3.     <button class="btn btn-success" (click)="addAddress()">New Address</button>
4.   </div>
5. </div>
6. <hr>
7. <div class="row">
8.   <div class="address-list col-md-3">
9.     @for (address of addresses; track $index) {
10.      <app-address-list-element
11.        [address]="address"
12.        (click)="select(address)">
13.      </app-address-list-element>
14.    }
```

```

15. </div>
16. <div class="col-md-9">
17.   @if (currentAddress !== null) {
18.     <app-address-view
19.       [address]="currentAddress"
20.       (fireDelete)="deleteCurrent()">
21.     </app-address-view>
22.   }
23. </div>
24. </div>

```

Les lignes 2 à 4 spécifient le bouton pour ajouter une nouvelle entrée d'adresse. Aux lignes 9 à 14, nous utilisons un bloc **for** pour boucler sur une collection d'adresses et créer un composant *Address List Element* pour chacune, en passant l'adresse actuelle en entrée. Les lignes 18 à 21 incorporent un composant *Address View*.

Classe de composant pour Address List

Ajoutez une propriété dans la classe *AddressList* pour la collection d'adresses.

```

1. addresses: AddressEntry[] = [];

```

Assurez-vous que la classe *AddressEntry* est ajoutée aux imports.

Address List Element

Génerez un composant d'élément d'adresse

([ng generate component address-list/address-list-element](#)). Un élément de liste d'adresses affiche un label identifiant l'entrée d'adresse. Nous utiliserons le nom complet construit à partir des champs *firstName* et *lastName*.

- Ajoutez la fonction suivante à la classe *AddressListElement* de `src/app/address-list/address-list-element/address-list-element.ts`.

```

1. getFullName(): string {
2.   return `${this.address.firstName}, ${this.address.lastName}`;
3. }

```

- Nous devons également déclarer une propriété *address* dans la classe *AddressListElement*. Cette instance sera transmise à l'instance de *AddressListElement* par le parent *AddressList* puisque les adresses doivent être détenues par cette classe.

Nous utilisons la liaison d'entrée (input binding) pour cela. Ajoutez la déclaration

```

1. @Input() address: AddressEntry | undefined;

```

à la classe *AddressListElement*. Vous devez également importer *Input* de `@angular/core` ainsi que *AddressEntry* de `./address-entry`.

- Nous souhaitons également modifier l'apparence d'un Address List Element lorsqu'il est sélectionné. Ajoutez la déclaration suivante à la classe `AddressListElement`.

```
1. selected = false;
```

Nous pourrions utiliser cette propriété pour définir les propriétés CSS du composant en fonction de leur statut sélectionné.

- Modifiez le modèle HTML d'élément de liste d'adresses (`src/app/address-list/address-list-element/address-list-element.html`) comme suit.

```
1. <button class="address-list-element ui positive"
2.     [ngClass]="{'address-list-element-selected': selected}">
3.     {{getFullName()}}
4. </button>
```

- Ajoutez une importation de la classe `NgClass` à la classe `AddressListElement`. Le nom de la classe doit être ajouté au tableau `imports` du décorateur `@Component`.

```
1. import {NgClass} from "@angular/common";
2.
3. @Component({
4.   selector: 'app-address-list-element',
5.   imports: [
6.     NgClass
7.   ],
8.   templateUrl: './address-list-element.html',
9.   styleUrls: ['./address-list-element.css']
10. })
11. ...
```

- Le style est contrôlé par la directive `ngClass` qui ajoute la propriété `address-list element-selected` lorsque la propriété `selected` est true.

Modifiez la feuille de style CSS de `AddressList Element` comme suit.

```
1. .address-list-element {
2.   border: 1px solid ;
3.   padding: 10px 2px;
4.   font-size: 16px;
5.   cursor: pointer;
6.   width: 200px;
7.   display: inline-block;
8. }
9. .address-list-element:active {
10.   color: green;
11. }
12. .address-list-element-selected {
13.   background-color: green;
14.   color: white;
```

```
15. font-weight: bold;
16. }
```

Composant Address View

Génerez un composant de vue d'adresse (Address View)
(ng generate component address-list/address-view).

Template Address View

Ce composant affiche un formulaire pour les entrées d'adresses. Modifiez le template HTML
(src/app/address-list/address-view/address-view.html) comme suit.

```
1. <form class="ui large form segment">
2.   <h3 class="ui header">Address Details</h3>
3.   <fieldset [disabled]='!edit ? 'disabled' : null">
4.     <div class="form-group">
5.       <label for="firstName">First Name</label>
6.       <input class="form-control" name="firstName" id="firstName" [(ngModel)]="address.firstName">
7.     </div>
8.     <div class="form-group">
9.       <label for="lastName">Last Name</label>
10.      <input class="form-control" name="lastName" id="lastName" [(ngModel)]="address.lastName">
11.    </div>
12.    <div class="form-group">
13.      <label for="phone">Phone Number</label>
14.      <input class="form-control" name="phone" id="phone" [(ngModel)]="address.phone">
15.    </div>
16.    <div class="form-group">
17.      <label for="email">Email</label>
18.      <input class="form-control" name="email" id="email" [(ngModel)]="address.email">
19.    </div>
20.    <div class="form-group">
21.      <label for="notes">Notes</label>
22.      <textarea class="form-control" rows="4" cols="60" name="notes" id="notes"
23.        [(ngModel)]="address.notes"></textarea>
24.    </div>
25.  </fieldset>
26.  <div class="btn-toolbar" role="toolbar">
27.    <button class="btn btn-danger" name="delete" (click)="delete()">Delete</button>
28.    @if (edit) {
29.      <button (click)="toggleEdit()" class="btn btn-secondary m-auto">Protect</button>
30.    } @else {
31.      <button (click)="toggleEdit()" class="btn btn-secondary m-auto">Edit</button>
32.    }
33.  </div>
34. </form>
```

Nous spécifions des champs de formulaire pour l'entrée d'adresse (lignes 6, 10, 14, 18, 22) et nous lions avec des propriétés dans la classe de composant.

Nous ajoutons également un bouton pour supprimer l'entrée (ligne 26) et un bouton pour basculer l'édition (lignes 27-33). L'événement cliquez sur le bouton *Delete* est géré par la fonction `delete` du composant. Afin d'afficher un bouton qui bascule de *Edit* à *Protect* lorsqu'on clique dessus, les lignes 27-31 spécifient deux éléments de bouton. Aux lignes 27-29, un bouton *Protect* est rendu lorsque la valeur `edit` est `true` aux lignes 29-31 un bouton *Edit* est rendu autrement.

Classe Address View

La classe du composant Address View, `AddressView` déclare les propriétés et les fonctions nécessaires au modèle HTML. Modifiez `AddressView` de `src/app/address-list/address-view/address-view.ts` comme suit.

```
1. import {Component, EventEmitter, Input, Output, OnInit} from '@angular/core';
2. import {AddressEntry} from '../address-entry';
3. import { NgIf } from '@angular/common';
4. import { FormsModule } from '@angular/forms';
5.
6. @Component({
7.   selector: 'app-address-view',
8.   templateUrl: './address-view.html',
9.   styleUrls: ['./address-view.css'],
10.  imports: [FormsModule, NgIf]
11. })
12. export class AddressView implements OnInit {
13.   @Input() address!: AddressEntry;
14.   @Output() fireDelete: EventEmitter<AddressEntry> = new EventEmitter();
15.   edit: boolean | undefined;
16.
17.   ngOnInit(): void {
18.     this.edit = true;
19.   }
20.
21.   toggleEdit(): void {
22.     this.edit = !this.edit;
23.   }
24.
25.   delete(): void {
26.     this.fireDelete.emit(this.address);
27.   }
28. }
```

Assurez-vous que les classes `FormsModule` ainsi que `NgIf` sont importés dans la classe `AddressView` (`src/app/address-list/address-view/address-view.ts`) et ajouté aux **imports** du décorateur `@Component`.

La liaison d'entrée (input binding) est utilisée à la ligne 13 pour transmettre l'adresse actuelle à modifier/afficher au composant. Un événement de sortie (output event - line 14) notifie le composant parent lorsque l'utilisateur clique pour supprimer l'adresse. L'événement est déclenché dans la fonction `delete` (lignes 25-27).

Style de Address View

Modifiez la feuille de style `AddressView` de `src/app/address-list/address-view/address-view.css` comme suit.

```
1. .address-view-elt {
2.   vertical-align: top;
3.   padding: 15px;
4. }
5. .address-view-btns {
6.   padding-top: 20px;
7. }
8. .address-view-elt {
9.   padding: 10px;
10. }
```

Ajout d'une Address Entry

Nous associons un gestionnaire au bouton *New Address* dans le composant *Address List* pour ajouter une *Address Entry*. Modifiez `AddressList` (`src/app/address-list/address-list.html`) pour qu'il soit comme suit.

```
1. <div class="row">
2.   <div class="col-md-12">
3.     <button class="btn btn-success" (click)="addAddress()">New Address</button>
4.   </div>
5. </div>
6. <hr>
7. <div class="row">
8.   <div class="address-list col-md-3">
9.     @for (address of addresses; track $index) {
10.      <app-address-list-element
11.        [address]="address"
12.        (click)="select(address)">
13.      </app-address-list-element>
14.    }
15.  </div>
16.  <div class="col-md-9">
17.    @if (currentAddress !== null) {
18.      <app-address-view
19.        [address]="currentAddress"
20.        (fireDelete)="deleteCurrent()">
21.      </app-address-view>
22.    }
23.  </div>
```

24. </div>

Nous avons ajouté un gestionnaire d'événements pour **click** à la ligne 3.

Modifiez la classe de composant pour ajouter les fonctions `addAddress` et `select`, ainsi que la propriété `currentAddress`.

```
1. @Component({
2.   ...
3.   imports: [NgIf, NgFor, AddressListElement, AddressView]
4. })
5. export class AddressList {
6.   addresses: AddressEntry[] = [];
7.   currentAddress: AddressEntry | null = null;
8.   constructor() { }
9.
10.  select(address: AddressEntry): void {
11.    this.currentAddress = address;
12.  }
13.
14.  addAddress(): void {
15.    const newAddress = new AddressEntry('New', 'Entry');
16.    this.addresses = [newAddress, ...this.addresses];
17.    this.select(newAddress);
18.  }
19. }
```

Modifiez (`src/app/address-list/address-list.html`) pour transmettre une instance sélectionnée de `AddressEntry` au composant `Address View`.

```
1. <div class="row">
2.   <div class="col-md-12">
3.     <button class="btn btn-success" (click)="addAddress()">New Address</button>
4.   </div>
5. </div>
6. <hr>
7. <div class="row">
8.   <div class="address-list col-md-3">
9.     @for (address of addresses; track $index) {
10.      <app-address-list-element
11.        [address]="address"
12.        (click)="select(address)">
13.      </app-address-list-element>
14.    }
15.   </div>
16.   <div class="col-md-9">
17.     @if (currentAddress !== null) {
18.       <app-address-view
19.         [address]="currentAddress"
```

```

20.     (fireDelete)="deleteCurrent()">
21.     </app-address-view>
22.   }
23. </div>
24. </div>
25. <div class="col-md-9">
26.   <app-address-view *ngIf="currentAddress !== null"
27.     [address]="currentAddress"
28.     (fireDelete)="deleteCurrent()"></app-address-view>
29. </div>
30. </div>

```

Nous associons également une fonction gestionnaire à l'événement `fireDelete` émis par le composant `Address View`. Ajoutez la fonction gestionnaire d'événement à la classe `AddressList`.

```

1. deleteCurrent(): void {
2.   this.addresses = this.addresses.filter((address: AddressEntry) => address !== this.currentAddress);
3.   this.currentAddress = null;
4. }

```

Cette fonction supprime l'entrée d'adresse de la collection d'entrées.

Notification de sélection

Lorsqu'un élément d'entrée d'adresse est sélectionné, l'élément précédemment sélectionné (le cas échéant) doit être désélectionné et son adresse associée remplace l'adresse précédente en tant qu'adresse dans le composant `Address View`. Nous ne pouvons pas utiliser les événements de sortie car les éléments d'adresse n'ont pas de relation parent-enfant entre eux. Nous utiliserons un observable pour communiquer.

Service de Notification

Générez un service avec la commande `ng generate service address-list/notification-service`. Modifiez `src/app/address-list/notification-service.ts` comme suit.

```

1. import { Injectable } from '@angular/core';
2. import { BehaviorSubject } from 'rxjs';
3. import { AddressEntry } from './address-entry';
4.
5. @Injectable()
6. export class NotificationService {
7.   // Observable for selected elements
8.   selectedElement = new BehaviorSubject<AddressEntry | null>(null);
9.   constructor() { }
10.
11.   public selectionChanged(address: AddressEntry): void {
12.     this.selectedElement.next(address);
13.   }
14. }

```

NotificationService déclare un Observable (un BehaviorSubject) à utiliser pour transmettre des objets AddressEntry lorsque la fonction selectionChanged est appelée.

Injection de Service

Nous utiliserons l'injecteur du composant AddressList pour injecter le service puisqu'il est parent de tous les composants Address Entry. Modifiez la classe AddressList comme suit.

```
1. import {Component, inject} from '@angular/core';
2. import {NgForOf, NgIf} from "@angular/common";
3. import {AddressEntry} from "../address-entry";
4. import {AddressListElement} from "../address-list-element/address-list-element";
5. import {AddressView} from "../address-view/address-view";
6. import {NotificationService} from "../notification-service";
7.
8. @Component({
9.   selector: 'app-address-list',
10.  imports: [
11.    NgIf,
12.    NgForOf,
13.    AddressListElement,
14.    AddressView
15.  ],
16.  providers: [NotificationService],
17.  templateUrl: '../address-list.html',
18.  styleUrls: ['../address-list.css']
19. })
20. export class AddressList {
21.   addresses: AddressEntry[] = [];
22.   currentAddress: AddressEntry | null = null;
23.   notificationService: Notification = inject(NotificationService);
24.
25.   select(address: AddressEntry): void {
26.     this.currentAddress = address;
27.     this.notificationService.selectionChanged(address);
28.   }
29.
30.   addAddress(): void {
31.     const newAddress = new AddressEntry('New', 'Entry');
32.     this.addresses = [newAddress, ...this.addresses];
33.     this.select(newAddress);
34.   }
35.
36.   deleteCurrent(): void {
37.     this.addresses = this.addresses.filter((address: AddressEntry) => address !== this.currentAddress);
38.     this.currentAddress = null;
39.   }
```

40. }

La ligne 17 spécifie l'injecteur de `AddressList` comme injecteur pour `NotificationService` en l'ajoutant au tableau `providers`. Le service est injecté dans la classe à la ligne 24 et lorsqu'une nouvelle entrée d'adresse est sélectionnée, nous appelons la fonction `selectionChanged` du service de notification avec l'entrée d'adresse nouvellement sélectionnée à la ligne 28. Cela pousse l'entrée d'adresse dans l'observable.

Notez que les **Signals** (<https://angular.dev/guide/signals>) introduits par Angular 17 pourraient être utilisés comme alternative aux **Observables**, pour implémenter le service de notification. Une telle implémentation avec des **Signals** est disponible à <https://github.com/stephanesome/address-book-signals>.

Notification de changement d'Entrée

Les composants `Address Entry` sont informés des changements d'adresse en s'abonnant à l'observable fourni par le service de notification. Modifiez `AddressListElement` comme suit.

```
1. import {Component, inject, Input, OnDestroy, OnInit} from '@angular/core';
2. import {AddressEntry} from '../address-entry';
3. import {NotificationService} from '../notification-service';
4. import {Subscription} from 'rxjs';
5. import { NgClass } from '@angular/common';
6.
7. @Component({
8.   selector: 'app-address-list-element',
9.   templateUrl: './address-list-element.html',
10.  styleUrls: ['./address-list-element.css'],
11.  imports: [NgClass]
12. })
13. export class AddressListElement implements OnInit, OnDestroy {
14.   @Input()
15.   address: AddressEntry | undefined;
16.   selected = false;
17.   subscription: Subscription | undefined;
18.   notificationService= inject(NotificationService);
19.
20.   ngOnInit(): void {
21.     this.subscription = this.notificationService.selectedElement.subscribe(newAddress => {
22.       this.selected = newAddress === this.address;
23.     });
24.   }
25.
26.   getFullName(): string {
27.     return `${this.address!.firstName}, ${this.address!.lastName}`;
28.   }
29.
30.   ngOnDestroy(): void {
```

```
31.   this.subscription!.unsubscribe();  
32. }  
33. }
```

Le service de notification est injecté à la ligne 19 et abonné aux lignes 22-24 dans une fonction de cycle de vie `ngOnInit`. Lorsqu'une nouvelle entrée est reçue, la propriété `selected property` prend comme valeur `true` si la nouvelle entrée est la même que celle détenue par le composant et `false` sinon. Le composant implémente également la fonction de suivie du cycle de vie (life-cycle hook) `ngOnDestroy` aux lignes 31-33, pour se désabonner de l'observable lorsque celui-ci est détruit.

Exercise

L'exercice suivant est le livrable pour le laboratoire. Complétez et enregistrez le code dans Github Classroom avant la date limite. Seul le travail de ce exercice sera évalué.

Implémentez une application Angular pour une liste de courses (shopping list). L'application doit afficher une interface similaire à la suivante.

brocoli	Ajouter
5 pommes	Supprimer
12 oeufs	Supprimer
1 pain	Supprimer

L'application présente un champ de texte et un bouton. Lorsqu'un utilisateur saisit un item dans le champ de texte et appuie sur le bouton, ce nouvel item doit être ajouté à une liste affichée des items de la liste de courses, avec un bouton associé permettant de supprimer l'item de la liste.

Votre implémentation devra compter deux composants Angular au moins : un composant avec le champs d'entrée de texte et le bouton 'Ajouter' ainsi qu'un composant présentant la liste des éléments et les boutons 'Supprimer' y associés.